This is a little more complicated in Minitest since you're loading ruby directly, and ruby doesn't handle file globs. You could create a rake task like this:

```
require 'rake/testtask'

Rake::TestTask.new(:fast) do |t|
  t.pattern = "test/{models,actions,values}/**/*_test.rb"
end
```

Or this way:

```
task :fast_tests do
  Dir.glob("test/{models,actions,values}/**/*_test.rb").each { |file| require file }
end
```

You can also do this directly from your shell if you think the rake tasks are too slow:

```
$ for file in test/{models,actions,values}/**/*_test.rb; do ruby $file; done
```

The side effect of the last approach is that each file will run and report separately, which may not be desirable.

Whatever you do, you probably want to make it a system alias, assign it a key shortcut, or do something else that's dependent on your own workflow. You want to be able to run these tests with as little effort as possible.

Recommendations for Faster Tests

The topic of Rails code structure and the attendant ability to write tests always winds up in a mixture of things everyone should try once, things I do regularly, things I wish I could do regularly, things I don't do but generally regret not doing, and so on. In that spirit, here are a few recommendations:

- My current practice is to move complex transaction logic out of controllers into workflow objects, complex creation logic out of models and into factory or workflow objects, and (somewhat less often) view logic into presenters—all of those are plain, non-Rails Ruby objects. David Heinemeier Hansson might say all of these lead to overly complicated code. That hasn't been my experience. You should try for yourself.
- I like to use SimpleDelegator to create a new object that has functionality that applies to an ActiveRecord object only some of the time—for example, logic about whether a user has access to a resource, or logic for purchasing an item, which is needed only during the purchase process itself.
- Where I can, I like to use immutable value objects—for instance, taking a start and end date from an object and making a DateRange class or